RESEARCH REPORT / 8 NOVEMBER 2026

GAP20: A 100M-token Transformer comparison

Architecture, mathematical formulation, language-model results, factual-recall probes and compilation benchmark

A four-layer fused-gate Transformer was compared with a standard four-layer Transformer on **100 million scored WikiText-103 next-token targets per model**. This report describes the completed experiment and its limits.

100M scored training targets per model	**19.91%** fewer parameters in trained GAP20
40.05 / 45.62 standard / GAP20 test perplexity	**+19.93%** reported compiled-vs-eager GAP throughput*

Finding. The widened GAP20 model uses fewer weights, but the standard model performs better on held-out language prediction. Neither tiny model shows reliable zero-shot retrieval of new prompt-defined facts. Compilation improved GAP20 throughput in one benchmark, but this is separate from the architecture comparison.

Copyright © 2026 GAP20 original author | rickeyernest1990@gmail.com

Code: PolyForm Noncommercial License 1.0.0. WikiText-103-derived data have separate source licensing. *Speed-up excludes compilation startup and is benchmark-specific.

Abstract

The completed experiment compares a standard decoder-only Transformer (337,280 parameters) with a wider GAP20 gated Transformer (270,128 parameters). Both used four attention blocks, four heads, a 96-token context, and a 2,048-piece SentencePiece vocabulary trained on WikiText-103 training text. Each processed the same 100 million nonrepeated scored targets under matched optimizer settings and data order. The GAP20 architecture replaces the standard expanded feed-forward network with a compact multiplicative SiLU gate.

On 24,576 matched test targets from 256 held-out windows, the standard model reached perplexity 40.05 and top-1 accuracy 30.34%; GAP20 reached 45.62 and 28.61%. The paired NLL difference (standard minus GAP20) was -0.1302 nats/token, with a 95% window-bootstrap interval [-0.1425, -0.1188]. The supplied synthetic fact-binding tests were close to chance for both models, and query-swap tests showed little sensitivity to which fact was requested. Separately, an extended CPU microbenchmark reported a 19.93% steady-state training throughput increase for torch.compile of GAP20 relative to eager GAP20, with noisy timing groups.

These results are specific to the small models and single training seed tested. The supplied archives were reviewed and the final checkpoints were inspected, but no independent 100M retraining or replicated hardware benchmark is claimed.

1. Research question and comparison

Can the GAP20 block reduce model parameters while retaining the language-model performance of an ordinary Transformer at a fixed number of scored training targets? The reported run deliberately **widens the GAP model**, spending some of the initial parameter saving on additional feature capacity. The experiment is therefore a practical design comparison, not a single-variable architectural ablation.

Design choice	Standard model	Trained GAP20
Layers / attention heads	4 / 4	4 / 4
Model width	64	72
Query and key size per head	16	22
Value size per head	16	18
Block transformation	attention + 4× GELU MLP	attention + multiplicative SiLU gate
Context / vocabulary	96 / 2,048	96 / 2,048
Trainable parameters	337,280	270,128

What is and is not being compared: equal source data, scored targets, batch, sequence length, and optimization schedule; **not** equal parameter count, floating-point work, or wall-clock training time.

2. Block mathematics

Both models process a sequence of token vectors using causal attention. At each position, attention produces a weighted combination of preceding token representations, with the future masked out.

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M_{\mathrm{causal}}\right)V$$

Q, K and V are learned query, key and value projections. The division by the square root of the key dimension stabilizes attention scores. The mask blocks future positions.

Standard Transformer block

$$z = x + W_A \, \mathrm{MHA}(\mathrm{LN}_1(x))$$

$$y = z + W_2 \, \mathrm{GELU}(W_1 \mathrm{LN}_2(z))$$

The standard block applies causal attention, an output projection, a residual connection, then a separate two-layer feed-forward network with a GELU activation and another residual connection.

GAP20 block

$$z = \mathrm{LN}_2(x + \mathrm{MHA}(\mathrm{LN}_1(x)))$$

$$y = x + W_d[z \odot \mathrm{SiLU}(W_G z)]$$

The GAP20 block normalizes the attention-enriched state z, uses a learned SiLU gate to scale each feature of z, projects it back to the model width, and adds the original input x. This is the tested GAP equation; widening the model changes tensor sizes, not the defining computation.

3. Parameter budgets and design provenance

Configuration	Width	Q/K per head	Parameters	Saving vs standard
Standard baseline	64	16	337,280	—
GAP20: compact configuration	64	16	221,568	34.31%
GAP20: trained widened configuration	72	22	270,128	19.91%

The compact configuration is obtained by applying the same block equations at width 64 and Q/K size 16: (337,280 - 221,568) / 337,280 = 34.31% fewer parameters. **This is a code-derived parameter calculation, not a completed 100M training outcome for that configuration.** The completed run uses width 72 / QK 22 and so gives (337,280 - 270,128) / 337,280 = 19.91% fewer parameters.

Because the architecture is configurable, varying width, number of heads, depth or Q/K dimension creates a different configuration of the GAP block. It does not, by itself, demonstrate invention of a new block. The repository preserves the original model implementation and supplies parameterized variants with attribution.

4. Dataset and experimental procedure

The training corpus was **WikiText-103**, a collection of Wikipedia article text. The supplied experiment used the user-provided WikiText-103 source archive and a **2,048-piece SentencePiece BPE tokenizer trained on training text only**. After tokenization, the supplied counts are 173,656,383 training BPE tokens, 365,278 validation tokens, and 414,862 test tokens.

Setting	Recorded experiment value
Training target	100,000,000 scored next-token positions per model
Example length	96 input tokens / 96 scored positions
Batch	Up to 32 nonoverlapping windows
Optimizer	AdamW, weight decay 0.01, gradient clipping 1.0
Learning rate	Peak 0.001; 200K-target warmup; cosine decay to 0.0001 across 100M
Random seed	20261108; matched shuffled training windows
Completed updates	32,600 optimizer steps per model
Official checkpoints	Every 5M scored targets; 20 per model
Held-out test	256 matched windows × 96 = 24,576 positions
Confidence interval	3,000 paired bootstrap resamples over test windows

A scored target is one next-BPE-token prediction, not one ordinary word. Source-file-disjoint validation and test splits were used. The same 256 test windows were evaluated at each checkpoint, so the 20 milestone values are correlated rather than independent test datasets.

5. Results on WikiText-103

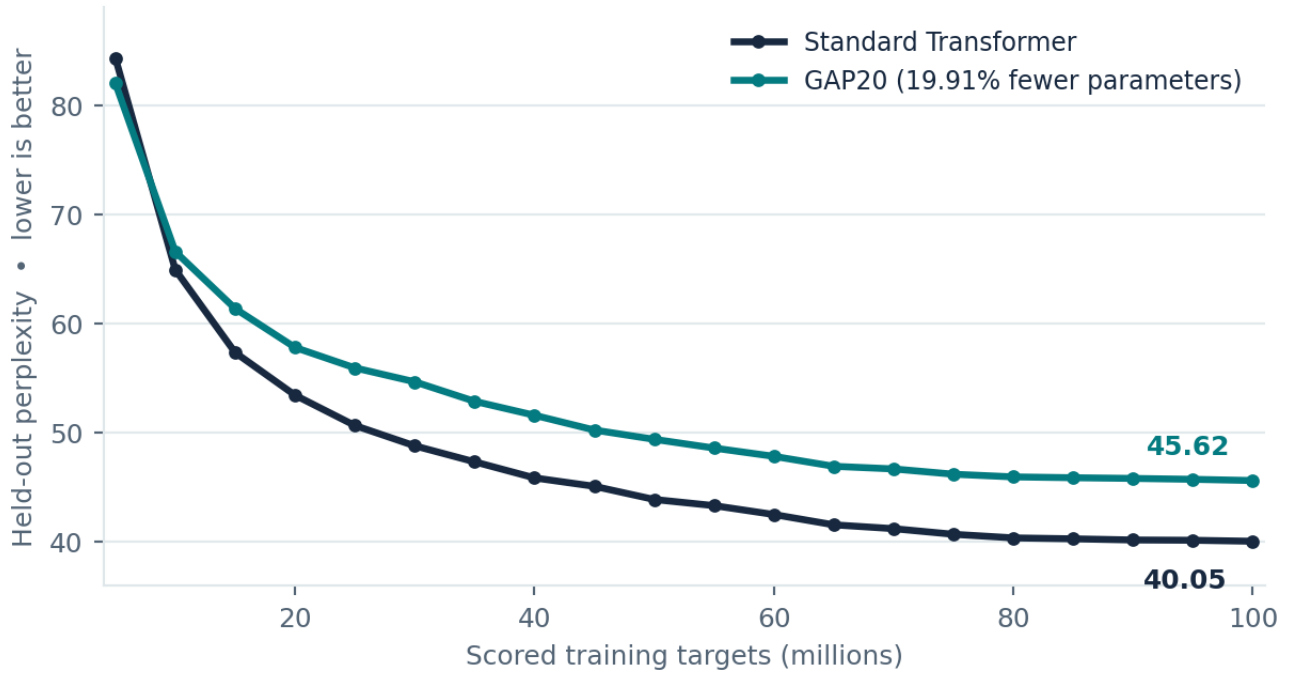


Figure 1. Held-out perplexity at every official 5M-token checkpoint, 5M through 100M. Lower is better. There is no separate 40M outcome to interpret as the final result.



Figure 2. Top-1 accuracy on the same matched held-out windows. Higher is better.

Final 100M metric	Standard	GAP20
Parameters	337,280	270,128
Test NLL (nats/token)	3.6902	3.8204
Test perplexity ↓	40.05	45.62
Test top-1 accuracy ↑	30.34%	28.61%

At the completed target, GAP20 test perplexity is 13.90% higher, and top-1 accuracy is 1.73 percentage points lower. Standard-minus-GAP20 mean test NLL is **-0.1302**, with paired 95% interval [-0.1425, -0.1188]; the negative sign favours standard. The intervals reflect resampling of these held-out windows, **not** variation over training seeds or datasets.

Detailed milestone values, original checkpoint metadata and per-window measurements are supplied as machine-readable files in the accompanying repository rather than duplicated in a 20-row table here.

6. Context copying and factual recall

A natural-text copying probe repeats a ten-token phrase with filler at separations of 0, 16, 48 and 72 tokens. Prediction loss on the repeated phrase is compared against a control containing a different earlier phrase. A positive benefit means the earlier phrase was useful, not that the model has human-like factual memory.

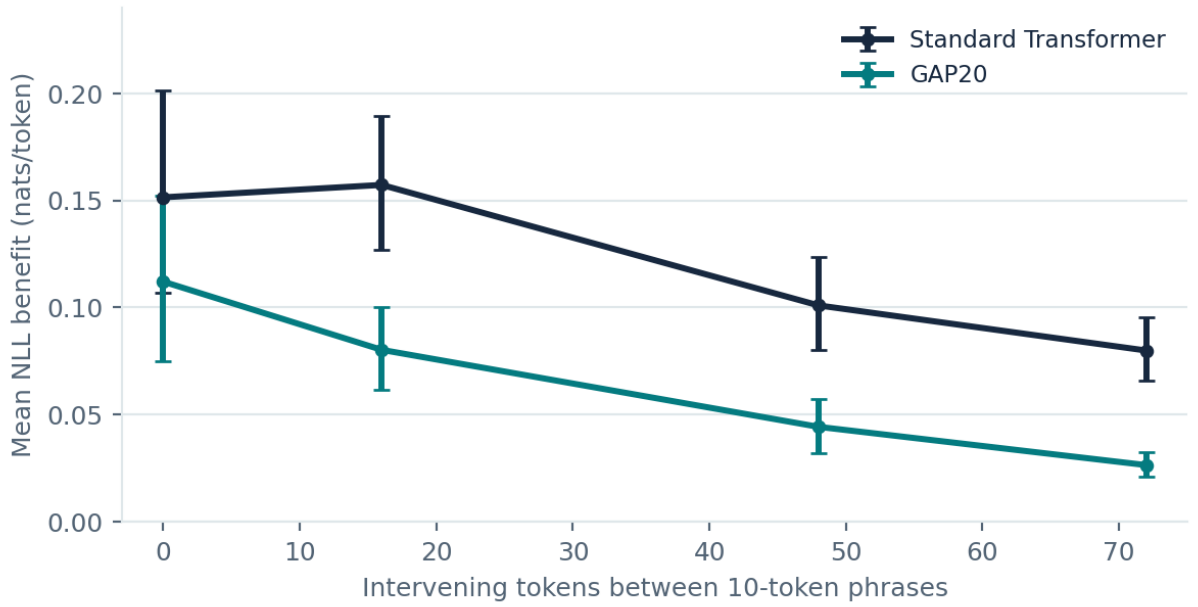


Figure 3. Copying benefit in nats/token, by intervening-token distance. The standard model benefits more at every tested distance.

Separate zero-shot tests used the completed checkpoints with no retraining. They supplied 2-, 4- or 8-fact name-to-value assignments, overwritten facts, and two-hop relationships. Each candidate answer was a single BPE token, and the model chose among candidate values already present in the prompt. The procedure does **not** measure free-form answers, memorized real-world knowledge or capabilities after task-specific fine-tuning.

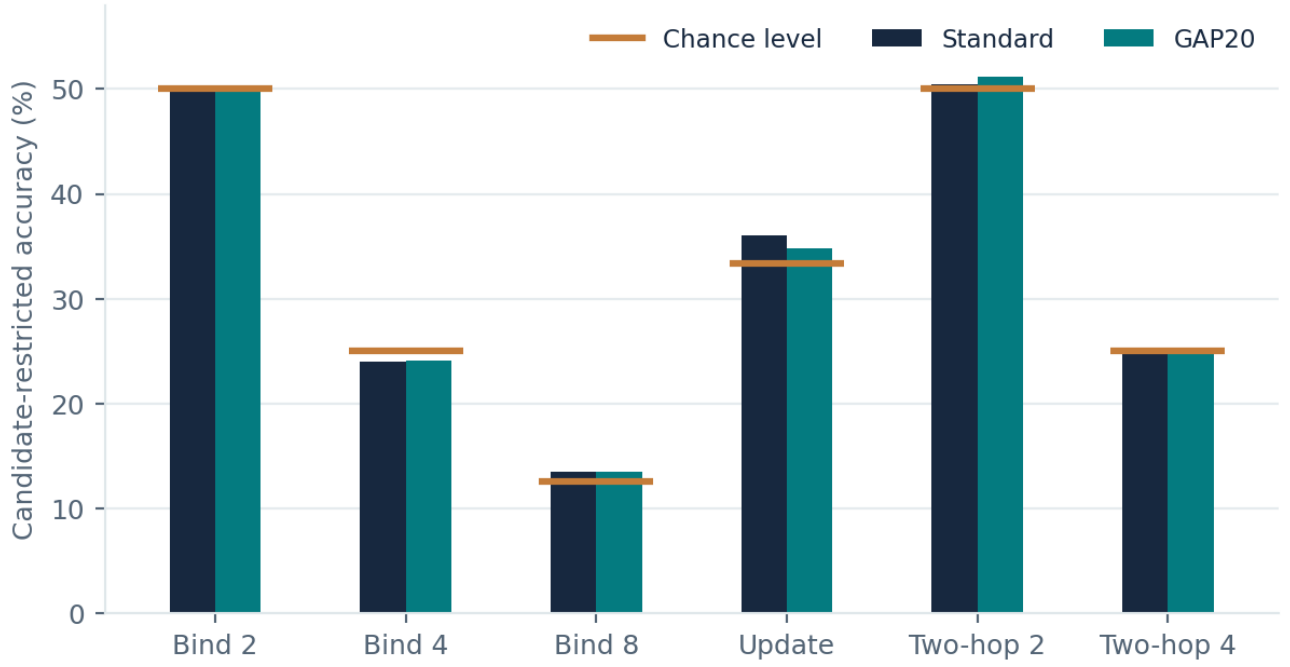


Figure 4. Candidate-restricted exact-match accuracy on original queries; horizontal orange lines show chance rates.

Task	Chance	Standard	GAP20
Bind 2 facts	50.0%	50.24%	49.80%
Bind 4 facts	25.0%	23.97%	24.07%
Bind 8 facts	12.5%	13.48%	13.48%

Task	Chance	Standard	GAP20
Update a fact	33.3%	36.08%	34.77%
Two-hop, 2 options	50.0%	50.44%	51.22%
Two-hop, 4 options	25.0%	24.85%	24.76%

There were 1,024 questions per task and format, two formats per task: 12,288 original and 12,288 query-swapped questions. Neither model consistently recovered entity-specific values. Swapping only the queried person changed the models' candidate selection infrequently (0% for both in eight-fact binding). This is consistent with answer preferences rather than robust fact retrieval. The apparently exact tie in eight-fact scores is a failure mode: both models tended to predict the same candidate within each format.

7. Full-model compilation benchmark

The separate speed experiment compares **the trained GAP20 model against itself**: eager PyTorch versus torch.compile(model, backend="inductor", fullgraph=True). On CPU with four intra-op threads, the benchmark used batch size 16 and sequence length 96. Each timed training step included forward prediction, cross-entropy, backward pass, gradient clipping and AdamW update. Compiler startup and warm-up were excluded from steady-state timings.

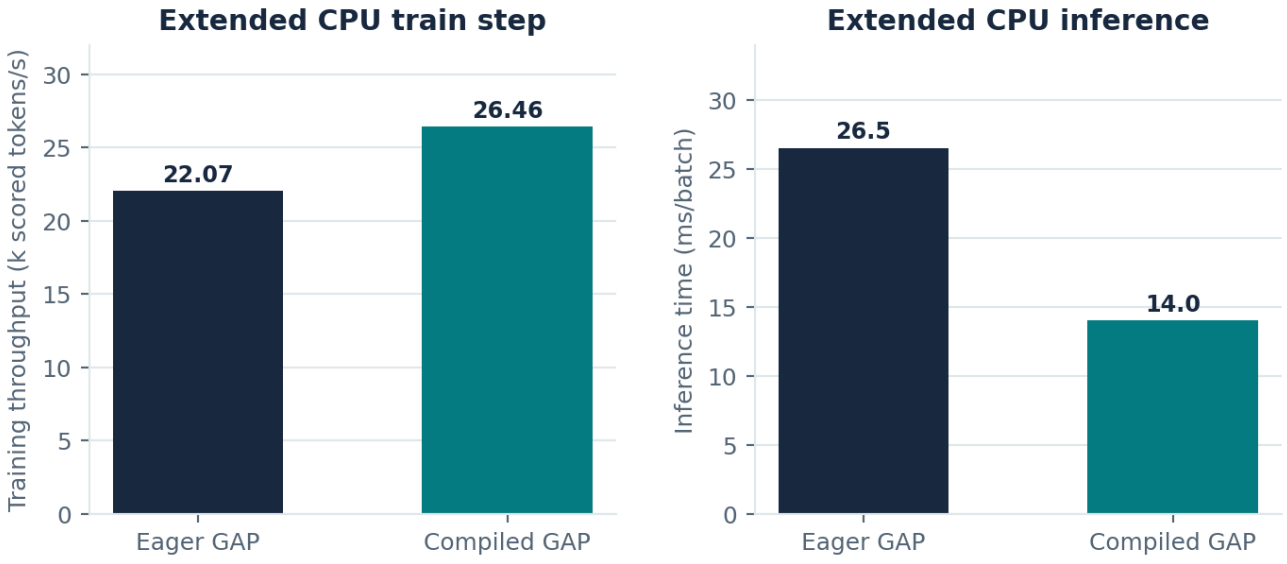


Figure 5. Extended CPU results from the submitted benchmark logs; illustrative of this particular timed run, not a universal speed claim.

Measure	Eager GAP20	Compiled GAP20
Median training time/step	0.06961 s	0.05804 s
Training throughput	22,067 tokens/s	26,464 tokens/s
Median inference time/batch	0.02654 s	0.01402 s
Training uplift	Baseline	+19.93%

Compilation cost is substantial: in the extended run the compiled first forward took about 15.94 seconds (and could be slower without cached code); this cost is excluded above. Some eager timing groups were especially noisy, so repeated alternating trials on an idle computer are required before making a stable claim. The original eager standard-vs-GAP benchmark showed similar step times; do not confuse compilation benefit with architectural speed superiority.

Core speed-up call (full runnable benchmark included in repository)

```
from models import LanguageModel
import torch

model = LanguageModel('gap20', gap_width=72, gap_qk_head=22)
# Load matching 100M checkpoint weights before benchmarking.
compiled_model = torch.compile(
    model, backend='inductor', fullgraph=True
)
# Time warmed forward + loss + backward + optimizer steps.
# Run eager and compiled in separate fresh processes.
```

8. Interpretation, reproducibility and limitations

What is supported: GAP20 is a working configurable fused-gate block; the 100M widened version has 19.91% fewer parameters but lower language-model quality than the standard model at this budget. A width-64 configuration mathematically has 34.31% fewer parameters. Neither fixed, pretrained system demonstrates dependable zero-shot factual binding in the submitted candidate tests. The measured GAP compilation throughput improvement warrants confirmation on additional machines.

What is not supported: broad claims of architecture superiority or impossibility; memory capability after adaptation; hardware-independent speedups; equal-compute comparison; results for the compact 34.31%-smaller configuration at 100M; results from independent training seeds. A 96-token context and 2,048-piece vocabulary limit the scope.

Public repository and reproduction scope

The linked public repository is intentionally limited to six files: this report, configurable model code, a runnable CPU torch.compile benchmark, README, software licence and copyright notice. WikiText-103 data, tokenizer, trained checkpoints, full evaluation logs and factual-memory test scripts are not distributed there. The numerical findings in this report come from the original supplied experiment archive, not an independent retraining or public six-file reproduction.

After installing PyTorch, run python models.py to display architecture parameter counts. Run python benchmark_compile.py --mode eager and --mode compiled in separate processes to collect new measurements. The benchmark defaults to randomly initialized weights; the historical 19.93% figure requires the original trained weights, hardware conditions and protocol. New measurements must be reported separately. The original 40M run is an intermediate checkpoint, not a final result.

Licensing and attribution

Required Notice: Copyright © 2026 GAP20 original author, rickeyernest1990@gmail.com. The project software is made available under the **PolyForm Noncommercial License 1.0.0** (polyformproject.org/licenses/noncommercial/1.0.0). The PDF report carries the author copyright notice; PolyForm is a software license, not a claim of ownership over general mathematical ideas. Noncommercial software use, modification and redistribution remain subject to the license conditions.

WikiText-103 and tokenized material are separately attributed to the WikiText dataset and underlying Wikipedia contributions. The supplied corpus licence file identifies **Creative Commons Attribution-ShareAlike 3.0**; it is not replaced by PolyForm. Consult the repository data notice before redistribution. Copyright of software protects code expression but does not automatically give exclusive ownership over a broad mathematical idea. The original block and its configuration lineage are documented to assist accurate attribution.

Contact: **rickeyernest1990@gmail.com** for permissions, collaboration or corrections.

Sources: Submitted 100M model code/checkpoints, 20-milestone held-out CSV/JSON, factual-memory probes, compile benchmark, and token archive. External references: PolyForm Noncommercial License 1.0.0 (polyformproject.org); Merity et al., Pointer Sentinel Mixture Models (2016); Salesforce WikiText dataset card. The supplied outputs were reviewed, not independently retrained.